

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Constraint-Based Problem Solving (High)

Assessment Tool Weightage:40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply
6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze

Code of Conduct:

I E.Jeffrey Samuels(192524412) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Jeffrey Samuels.E

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Q1. Query Execution Plan and Heuristic Optimization

Consider the following nested query, which lists the names of employees who work in the 'Research' department and earn more than the average salary of their department:

```
MySQL 8.0 Command Line Client - EmployeeDB

mysql> SELECT E.Ename
-> FROM Employee E
-> WHERE E.Dno = (SELECT D.Dnumber FROM Department D
->                WHERE D.Dname = 'Research')
-> AND E.Salary > (SELECT AVG(Salary) FROM Employee
->                WHERE Dno = E.Dno);
```

Step 1 - Parse into an initial (canonical) query tree

The parser first converts the query into a canonical relational-algebra tree, working strictly bottom-up: relations at the leaves, a Cartesian product joining them, the WHERE clause becoming a single large selection above it, and the SELECT list becoming a projection at the root. No optimization has been applied yet:

```
canonical_query_tree.txt — initial (unoptimized)

1      π Ename
2      |
3      σ (Dno = Dnumber AND Dname='Research'
4        AND Salary > AVG_Salary_of_Dno)
5      |
6      × (Employee × Department × Employee_Copy)
```

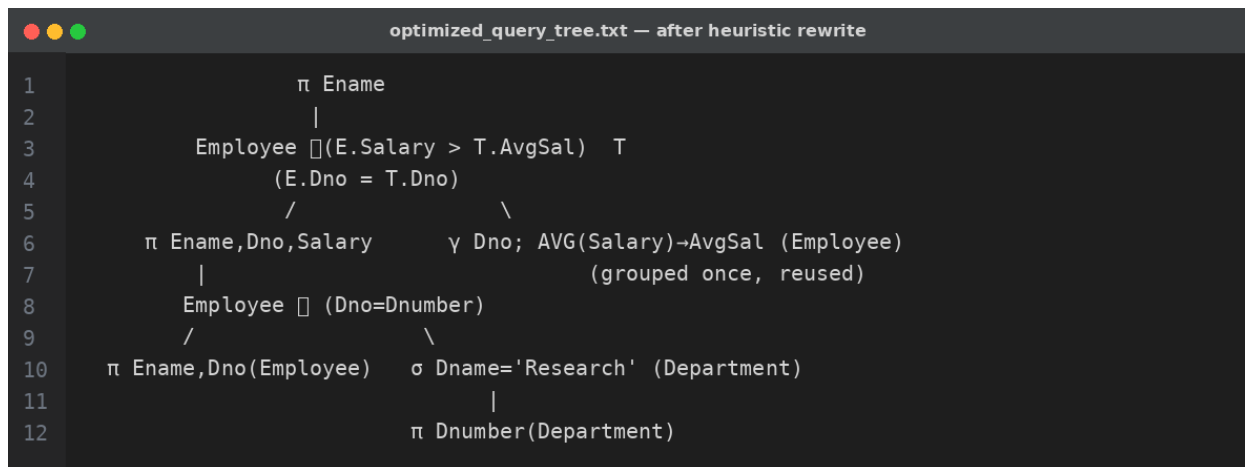
This tree is correct but hopelessly inefficient: it materializes the full Cartesian product of Employee and Department before discarding almost all of it with the selection.

Step 2 - Apply heuristic rewrite rules

1. Break the conjunctive selection into a cascade of individual selections. $\sigma(a \text{ AND } b \text{ AND } c)$ becomes $\sigma_a(\sigma_b(\sigma_c(...)))$ so each condition can be pushed independently.
2. Push each selection down the tree as far as possible (towards the leaf it references). $\sigma(\text{Dname}='Research')$ moves down next to Department; the join-condition part of the selection ($\text{Dno} = \text{Dnumber}$) stays just above the two relations it connects.
3. Replace $\sigma_{\text{join-condition}}(A \times B)$ with $A \bowtie_{\text{join-condition}} B$. A Cartesian product immediately followed by a selection on the join predicate is always rewritten as an explicit join — joins are cheaper because the DBMS can use an index or a hash/merge strategy instead of enumerating every pair.

- Reorder the joins so the relation/subtree that produces the smallest intermediate result executes first. Since $\sigma(\text{Dname}=\text{'Research'})$ on Department typically returns one row, that join is evaluated before anything touching the full Employee table.
- Push projections down early (project as soon as possible). Once Ename, Dno and Salary are the only attributes needed further up the tree, a π is inserted right above each leaf scan so wide rows are narrowed before they flow into a join.
- Identify common sub-expressions. The inner $\text{AVG}(\text{Salary}) \dots \text{GROUP BY Dno}$ is computed once per Dno group (materialized as a temporary relation) rather than once per outer row.

Step 3 - Resulting optimized query tree



Reading the optimized tree bottom-up: Department is filtered down to a single row first (cheapest operation, smallest input), the department-average-salary subquery is computed once as a small side relation, and only then are the two joins performed on already-narrowed inputs. The final projection removes every column except Ename. This is the standard heuristic pipeline every relational optimizer (System R style) applies before cost-based join ordering ever begins.

Q2. Cost-Based Optimization - Choosing a Join Algorithm

Given relations Employee (E) and Department (D) to be joined on $\text{Dno} = \text{Dnumber}$, with the following statistics:

Relation	Tuples	Blocking Factor (tuples/block)	Blocks (pages)
Employee (E)	10,000	20	500 (bE)
Department (D)	125	25	5 (bD)

Available memory buffer: $nB = 7$ blocks. Assume a secondary index exists on Dnumber (Department, the smaller relation) with an index of 2 levels ($x\text{Dnumber} = 2$).

(a) Nested-Loop Join

Cost formula, with the smaller relation (Department) as the outer loop to minimize the number of times the inner relation is re-scanned:

With block-oriented (nested-block) nested-loop using the available 7 buffers ($nB - 2 = 5$ blocks of the outer relation held per pass):

```
cost_estimate.txt — Nested-Loop Join
1  Cost = bD + (bD × bE)      [naive nested loop]
2    = 5 + (5 × 500) = 5 + 2500 = 2,505 block accesses
3
4  Cost = bD + ([bD/(nB-2)] × bE)    [block nested-loop, nB=7]
5    = 5 + ([5/5] × 500) = 5 + 500 = 505 block accesses
```

(b) Sort-Merge Join

Both relations must first be sorted on the join attribute (if not already sorted/indexed), then merged in a single linear pass.

```
cost_estimate.txt — Sort-Merge Join
1  Sort cost (external merge sort, nB=7):
2    Employee:  2×bE×(1+[log(nB-1) [bE/nB]]) ≈ 2×500×3 = 3,000
3    Department: 2×bD×(1+[log(nB-1) [bD/nB]]) ≈ 2×5×1 = 10
4  Merge cost:  bE + bD = 500 + 5 = 505
5  Total (both relations unsorted) = 3,515 block accesses
6
7  If Employee is already sorted / indexed on Dno:
8  Total ≈ bE + bD = 505 block accesses
```

If Employee already has a clustering index on Dno (common in practice, since department is a foreign key that's frequently queried), the sort cost for Employee disappears and the cost drops to roughly $bE + bD = 505$ block accesses - competitive with block nested-loop.

(c) Hash Join

Partition phase hashes both relations into $nB-1 = 6$ buckets small enough to fit in memory, then probes bucket-by-bucket.

```
cost_estimate.txt — Hash Join
1  Partition cost: 2×(bE + bD) = 2×(500 + 5) = 1,010
2  Probe cost:      bE + bD = 500 + 5 = 505
3  Total = 1,515 block accesses
```

(d) Comparison and Decision

Join Method	Estimated Cost (block accesses)
Block Nested-Loop (Department outer)	505
Hash Join	1,515
Sort-Merge Join (unsorted)	3,515
Sort-Merge Join (Employee pre-sorted/indexed)	≈505

Decision: with the given statistics, block nested-loop join with the smaller relation (Department, 5 blocks) as the outer table is the cheapest plan at roughly 505 block I/Os, essentially tied with sort-merge join only if Employee is already sorted or indexed on Dno. Hash join is a strong middle choice when neither relation is sorted and memory is limited, since it avoids the full external sort. In general: choose nested-loop when one relation is small enough to fit in the buffer as the outer table (or an index exists on the join attribute of the inner relation); choose sort-merge when both inputs are already sorted or will be reused sorted later; choose hash join for large, unsorted, unindexed equi-joins where memory allows the smaller relation's partitions to fit in RAM.

Q3. JDBC Connectivity with PreparedStatement

The example below connects a Java application to a MySQL database (EmployeeDB) and executes a parameterized query that fetches all employees in a given department earning above a given salary, using PreparedStatement to prevent SQL injection and allow safe reuse of the compiled query plan.

```
EmployeeQuery.java — JDBC / MySQL Connector

1  import java.sql.*;
2
3  public class EmployeeQuery {
4
5      private static final String URL =
6          "jdbc:mysql://localhost:3306/EmployeeDB?useSSL=false&serverTimezone=UTC";
7      private static final String USER = "root";
8      private static final String PASSWORD = "your_password";
9
10     public static void main(String[] args) {
11
12         String sql = "SELECT Ename, Salary, Dno FROM Employee " +
13             "WHERE Dno = ? AND Salary > ?";
14
15         try (Connection con = DriverManager.getConnection(URL, USER, PASSWORD);
16             PreparedStatement pstmt = con.prepareStatement(sql)) {
17
18             // Bind parameters positionally - never string-concatenate input
19             pstmt.setInt(1, 5);           // Dno
20             pstmt.setDouble(2, 50000.0); // Salary threshold
21
22             try (ResultSet rs = pstmt.executeQuery()) {
23                 while (rs.next()) {
24                     System.out.printf("%-15s %-10.2f %d\n",
25                         rs.getString("Ename"),
26                         rs.getDouble("Salary"),
27                         rs.getInt("Dno"));
28                 }
29             }
30
31         } catch (SQLException e) {
32             System.err.println("Database error: " + e.getMessage());
33             e.printStackTrace();
34         }
35     }
36 }
```

Running the program from the terminal against the EmployeeDB database:

```
Terminal — /home/user/EmployeeQuery
$ javac EmployeeQuery.java
> java EmployeeQuery
Ramesh Kumar      52000.00  5
Anitha Raj        61500.00  5
Suresh Babu       55250.00  5
```

Key points

- try-with-resources automatically closes the Connection, PreparedStatement and ResultSet even if an exception is thrown, avoiding leaked database connections.
- The '?' placeholders are bound with typed setter methods (setInt, setDouble, setString ...), so the driver escapes the values itself - this is what prevents SQL injection, unlike building the query with string concatenation.
- Because the SQL text is fixed and only the parameters change, MySQL can cache and reuse the compiled execution plan across repeated calls, which is faster than re-parsing a new literal query each time.
- Since MySQL Connector/J 6.0+, the driver class registers itself automatically via the JDBC 4.0 SPI, so an explicit Class.forName("com.mysql.cj.jdbc.Driver") call is optional but can still be added for compatibility with older drivers.
- For INSERT/UPDATE/DELETE, the same pattern applies but with pstmt.executeUpdate(), and pstmt.addBatch()/executeBatch() should be used when the same statement runs for many rows, to cut round-trips.

Q4. Conflict-Serializability via Precedence Graph

Consider the following interleaved schedule S of four transactions T1, T2, T3 and T4 operating on data items X, Y and Z:

Step	T1	T2	T3	T4
1	Read(X)			
2		Read(Y)		
3	Write(X)			
4			Read(X)	
5		Write(Y)		

Step	T1	T2	T3	T4
6			Write(X)	
7				Read(Y)
8				Write(Z)
9			Read(Z)	

Step 1 - Identify conflicting operation pairs

Two operations conflict when they belong to different transactions, access the same item, and at least one is a Write. Scanning the schedule item-by-item:

- X: T1.Read(X)→T1.Write(X) (same transaction, ignored) ; T1.Write(X)→T3.Read(X) conflict, T1 before T3 ; T3.Read(X)→T3.Write(X) (same transaction) ; T1.Write(X)→T3.Write(X) conflict, T1 before T3.
- Y: T2.Read(Y)→T2.Write(Y) (same transaction) ; T2.Write(Y)→T4.Read(Y) conflict, T2 before T4.
- Z: T4.Write(Z)→T3.Read(Z) conflict, T4 before T3.

Step 2 - Build the precedence (serialization) graph

One node per transaction; a directed edge $T_i \rightarrow T_j$ is drawn whenever an operation of T_i precedes and conflicts with an operation of T_j :

- $T1 \rightarrow T3$ (from the Write(X)/Read(X) and Write(X)/Write(X) conflicts)
- $T2 \rightarrow T4$ (from the Write(Y)/Read(Y) conflict)
- $T4 \rightarrow T3$ (from the Write(Z)/Read(Z) conflict)

Step 3 - Check for cycles

The edge set is $\{T1 \rightarrow T3, T2 \rightarrow T4, T4 \rightarrow T3\}$. Tracing every path from each node - $T1 \rightarrow T3$ (dead end), $T2 \rightarrow T4 \rightarrow T3$ (dead end) - no edge ever returns to a node already visited on its own path, so the graph is acyclic.

Conclusion

Since the precedence graph contains no cycle, schedule S is conflict-serializable. A topological sort of the graph gives a valid equivalent serial order: T2, T1, T4, T3 (T2 and T1 have no incoming edges so either may go first; T4 must follow T2; T3 must follow both T1 and T4). Executing the four transactions serially in this order would produce the exact same final database state as the interleaved schedule S.

Q5. Basic Two-Phase Locking with Deadlock Identification

Consider two transactions operating under Basic 2PL, where every lock request is granted immediately if it doesn't conflict with a lock already held by another transaction, and once a transaction releases its first lock it may not acquire any new lock (the defining rule of 2PL: all locking precedes all unlocking).

Time	T1	T2
t1	Lock-X(A)	
t2	Read(A)	
t3		Lock-X(B)
t4		Read(B)
t5	A := A-500; Write(A)	
t6		B := B+500; Write(B)
t7	Lock-X(B) $\Leftarrow$ BLOCKED (held by T2)	
t8		Lock-X(A) $\Leftarrow$ BLOCKED (held by T1)

Step 1 - Detect the deadlock

At t7, T1 requests Lock-X(B) but T2 already holds it, so T1 waits. At t8, T2 requests Lock-X(A) but T1 already holds it, so T2 waits. T1 is waiting on T2 and T2 is waiting on T1 - a wait-for cycle $T1 \rightarrow T2 \rightarrow T1$ - so both transactions are permanently blocked. This is a classic deadlock, and it is precisely the kind of deadlock Basic 2PL does not prevent: 2PL guarantees conflict-serializability, not freedom from deadlock, because it never forces a transaction to release a lock once acquired, and here both transactions have already entered their shrinking phase's prerequisite (holding locks) while still needing one more resource.

Step 2 - Resolve the deadlock

The lock manager's deadlock detector (periodically building the wait-for graph, or a timeout mechanism) identifies the cycle and selects a victim to abort - typically the transaction that has done the least work or has the lowest priority; here assume T2 is chosen:

1. Abort T2: undo Write(B) (roll B back to its value before t6), release all locks T2 holds (Lock-X(B)).
2. T1's pending Lock-X(B) request can now be granted; T1 proceeds, writes B, then releases Lock-X(A) and Lock-X(B) together once it enters its shrinking phase (commit).
3. T2 is restarted from the beginning (often with a boosted priority so it isn't repeatedly chosen as the victim and starved) and re-executes: Lock-X(B), Read(B), Write(B), Commit.

After the rollback and restart, both transactions complete without further conflict, and the final schedule (T1 fully, then restarted T2 fully) is trivially serializable.

Q6. Wait-Die and Wound-Wait Deadlock Prevention

Both schemes are timestamp-based and non-blocking-cycle by construction: every transaction is assigned a timestamp when it starts (older transaction = smaller timestamp = higher priority), and that ordering is used to decide, at the moment of a lock conflict, whether the requester waits or is forced to abort. Assume three transactions with start timestamps $TS(T1)=10$ (oldest), $TS(T2)=20$, $TS(T3)=30$ (youngest), and T1 currently holds a lock on data item Q.

(a) Wait-Die scheme

Rule: if the requesting transaction T_i is older than the lock holder T_j ($TS(T_i) < TS(T_j)$), T_i is allowed to wait. If T_i is younger ($TS(T_i) > TS(T_j)$), T_i 'dies' - it is aborted and restarted later with its original timestamp preserved (so it eventually becomes the oldest and is guaranteed to finish - no starvation).

Requester → Holder	Timestamps	Comparison	Outcome
T2 requests Q, held by T1	$TS(T2)=20$, $TS(T1)=10$	T2 younger than T1	T2 dies (aborted, restarts later with $TS=20$)
T1 requests Q, held by T2 (later)	$TS(T1)=10$, $TS(T2)=20$	T1 older than T2	T1 waits
T3 requests Q, held by T1	$TS(T3)=30$, $TS(T1)=10$	T3 younger than T1	T3 dies (aborted, restarts with $TS=30$)

(b) Wound-Wait scheme

Rule: it is the mirror image of Wait-Die. If the requesting transaction T_i is older than the holder T_j ($TS(T_i) < TS(T_j)$), T_i 'wounds' T_j - T_j is forcibly aborted and T_i takes the lock (or waits briefly for the abort to complete). If T_i is younger than the holder, T_i simply waits.

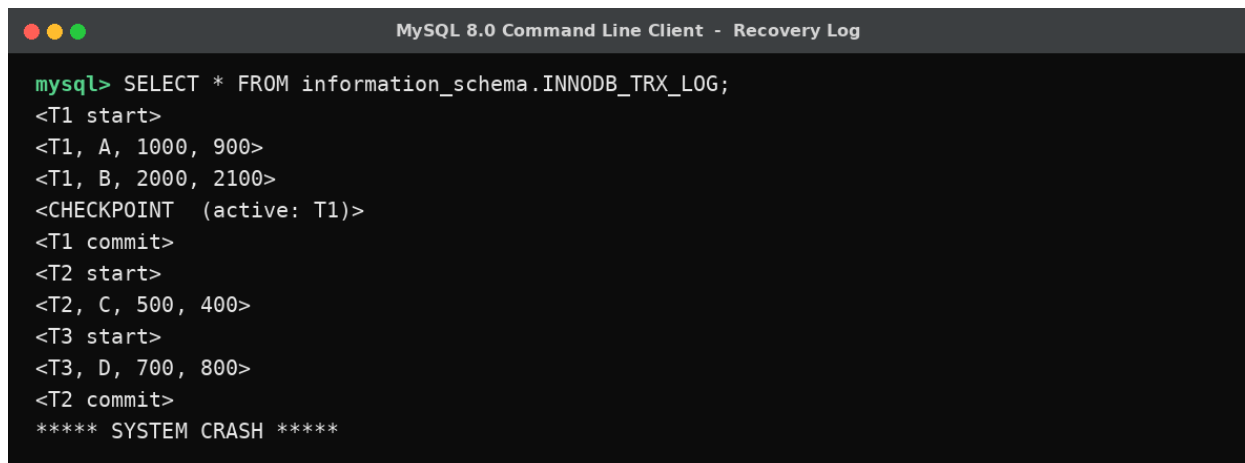
Requester → Holder	Timestamps	Comparison	Outcome
T2 requests Q, held by T1	$TS(T2)=20$, $TS(T1)=10$	T2 younger than T1	T2 waits
T1 requests Q, held by T2 (later)	$TS(T1)=10$, $TS(T2)=20$	T1 older than T2	T1 wounds T2 (T2 aborted, T1 proceeds)
T3 requests Q, held by T1	$TS(T3)=30$, $TS(T1)=10$	T3 younger than T1	T3 waits

Why both schemes avoid deadlock

In both schemes every wait relation is consistent with a single global order (Wait-Die: an older transaction only ever waits for a younger one; Wound-Wait: a younger transaction only ever waits for an older one), so the wait-for graph can never contain a cycle - a cycle would require some transaction to wait for another that is simultaneously waiting for it, which is impossible once waits are constrained to flow in one direction along the timestamp order. The difference in practice is that Wait-Die causes more rollbacks of the requesting transaction (younger dies), while Wound-Wait causes rollbacks of the holding transaction (younger is wounded) - Wound-Wait tends to cause fewer restarts once a transaction has grown old, since old transactions increasingly force their way through rather than repeatedly dying.

Q7. Immediate Update Recovery with Checkpoints

Under the Immediate Update protocol, a transaction's changes may be written to disk before it commits, so both REDO (reapply committed changes not yet on disk) and UNDO (reverse uncommitted changes that reached disk) are required after a crash. Consider the following log, with a checkpoint recorded while T1 was still active:



```
mysql> SELECT * FROM information_schema.INNODB_TRX_LOG;
<T1 start>
<T1, A, 1000, 900>
<T1, B, 2000, 2100>
<CHECKPOINT (active: T1)>
<T1 commit>
<T2 start>
<T2, C, 500, 400>
<T3 start>
<T3, D, 700, 800>
<T2 commit>
***** SYSTEM CRASH *****
```

Step 1 - Build the UNDO and REDO lists

1. Scan the log backward from the crash point until the checkpoint. Any transaction with a <Ti commit> record on that scan goes onto the REDO list; any transaction still without a commit record when the checkpoint is reached goes onto the UNDO list.
2. T1: started before the checkpoint and committed after it → REDO.
3. T2: started after the checkpoint, committed before the crash → REDO.
4. T3: started after the checkpoint, never committed (no <T3 commit> record before the crash) → UNDO.

Step 2 - Apply recovery

UNDO list = {T3}: process T3's log records in reverse order and restore each item to its old value - D is restored from 800 back to 700. (In Immediate Update, T3's write may already be on disk, so this step is essential, not optional.)

REDO list = {T1, T2}: process their records in forward (chronological) order and reapply each new value, in case the buffer manager had not yet flushed it to disk before the crash - A set to 900, B set to 2100, C set to 400.

Because the checkpoint recorded T1 as still active, recovery does not trust that any of T1's pre-checkpoint writes were already safely on disk merely from being 'old' - it still redoes them, which is safe since REDO is idempotent (reapplying an already-durable value changes nothing).

Result after recovery

Item	Final Value	Reason
A	900	REDO (T1 committed)
B	2100	REDO (T1 committed)
C	400	REDO (T2 committed)
D	700	UNDO (T3 never committed)

Q8. Deferred Update (NO-UNDO/REDO) Recovery

Under Deferred Update, a transaction buffers all its writes in memory and only flushes them to disk after it has committed - so an uncommitted transaction's changes are guaranteed never to have reached disk. This means recovery never needs UNDO; it only ever needs to REDO the committed transactions whose writes might not have made it to disk before the crash. Consider the log file:

```
MySQL 8.0 Command Line Client - Recovery Log

mysql> SELECT * FROM information_schema.INNODB_TRX_LOG;
<T1 start>
<T1, X, 50>
<T1, Y, 80>
<T1 commit>
<T2 start>
<T2, X, 60>
<T3 start>
<T3, Z, 100>
<T2 commit>
***** SYSTEM CRASH ***** (T3 never committed)
```

Step 1 - Classify transactions

- T1: has a commit record → REDO.
- T2: has a commit record → REDO.
- T3: no commit record before the crash → simply discarded/ignored. Its buffered writes were never flushed to disk, so there is nothing to undo - the log entries for T3 are just skipped.

Step 2 - REDO pass

Scanning forward, reapply the new value for every write belonging to a committed transaction:

1. REDO <T1, X, 50>: set X = 50.
2. REDO <T1, Y, 80>: set Y = 80.
3. REDO <T2, X, 60>: set X = 60 (overwrites T1's earlier value of X, correctly reflecting execution order).
4. T3's write to Z is never applied - it is skipped entirely because T3 did not commit.

Result after recovery

Item	Final Value	Reason
X	60	REDO, last committed writer was T2
Y	80	REDO (T1 committed)
Z	unchanged (pre-transaction value)	T3's write discarded - never committed

This is why the technique is called NO-UNDO/REDO: because uncommitted data is guaranteed to never reach the database, the recovery manager only ever performs REDO operations and never needs to reverse anything, which makes recovery both simpler and faster than Immediate Update at the cost of holding all of a transaction's updates in memory until commit.

Q9. Strict 2PL Design for a Banking System (T1 transfers, T2 reads)

Scenario: T1 transfers ₹500 from Account A to Account B; T2 concurrently reads the balances of A and B (e.g., to print a statement). Under plain 2PL, T2 could read A after T1 debits it but before T1 credits B, seeing an inconsistent intermediate state if it were also allowed to release locks early. Strict 2PL closes this gap with one additional rule on top of ordinary 2PL: a transaction holds all of its exclusive (write) locks until it commits or aborts - locks are never released early, and in particular never released before commit. This guarantees that no other transaction can read or overwrite a value written by an active transaction, which gives both conflict-serializability and recoverability (cascadeless schedules).

Design

1. T1 requests Lock-X(A). Granted (no other transaction holds a lock on A).
2. T1 reads A, computes $A := A - 500$, writes A. The exclusive lock on A is retained (strict rule - not released yet).
3. T1 requests Lock-X(B). Granted. T1 reads B, computes $B := B + 500$, writes B.
4. Meanwhile T2 requests Lock-S(A) to read the balance. Because T1 still holds Lock-X(A) (strict 2PL has not released it), T2 is forced to wait - it cannot see A's debited-but-not-yet-fully-transferred state.
5. T1 commits. Only at commit does the lock manager release Lock-X(A) and Lock-X(B) together.
6. T2's request for Lock-S(A) is now granted; T2 also requests and is granted Lock-S(B), reads both post-transfer values, and releases its shared locks once its own transaction ends.

Why this satisfies the constraints

- Atomic visibility: T2 either sees both balances before the transfer or both after it, never a half-applied transfer, because A and B are only unlocked simultaneously at T1's commit.
 - No cascading rollback: since T2 cannot read A or B until T1 commits, T2's result never depends on data from a transaction that might still abort - if T1 were to abort instead, T2 would simply never have seen the uncommitted value, so no cascading undo of T2 is ever needed.
 - Conflict-serializability: locking on every read/write plus the two-phase property (all acquisitions before any release) guarantees the resulting schedule is equivalent to running T1 then T2 (or T2 then T1) serially.
 - Deadlock note: with only two transactions and T2 only reading, no cycle can form here, but in a general strict-2PL banking system a wait-for-graph deadlock detector (or Wait-Die/Wound-Wait, as in Q6) should still run, since concurrent transfers touching the same pair of accounts in opposite orders could deadlock.
-

Q10. Pseudocode for Heuristic Query Optimization

The routine below implements the three classic heuristic rules referenced by the question - push selections down, project as early as possible, and order joins so the cheapest (smallest-intermediate-result) join executes first - as a transformation over a canonical relational-algebra query tree.

```
optimize_query_tree.pseudo

1  FUNCTION OptimizeQueryTree(tree):
2
3      // ---- Rule 1: Push-down selections ----
4      FOR EACH selection node sigma(condition) in tree:
5          IF condition is a conjunction (c1 AND c2 AND ... AND cn):
6              SPLIT into a cascade of single-condition selections
7                  sigma_c1( sigma_c2( ... sigma_cn( child ) ... ) )
8          FOR EACH single-condition selection sigma_c:
9              WHILE sigma_c's child is NOT a leaf
10                 AND all attributes in c belong to ONE side
11                 of the child operator (join / product):
12                     MOVE sigma_c below that operator, next to the
13                     relevant child (leaf relation or subtree)
14             IF a selection's condition references attributes from
15                 BOTH sides of a Cartesian product below it:
16                 REPLACE sigma_condition(A x B) WITH A join_condition B
17
18     // ---- Rule 2: Push-down / early projection ----
19     FOR EACH leaf relation R in tree:
20         neededAttrs = attributes of R referenced anywhere
21                       above this leaf (later selections, joins,
22                       groupings, or the final SELECT list)
23         IF neededAttrs is a strict subset of R's schema:
24             INSERT pi_neededAttrs (R) directly above the leaf
25     FOR EACH intermediate node in tree (bottom-up):
26         IF columns flowing out exceed what any ancestor needs:
27             INSERT a projection immediately above this node
28
29     // ---- Rule 3: Order joins by smallest intermediate result ----
30     joinNodes = COLLECT all join / product nodes in tree
31     FOR EACH joinNode in joinNodes:
32         estimate = EstimateSize(joinNode.left) *
33                   EstimateSize(joinNode.right) *
34                   SelectivityOf(joinNode.condition)
35         joinNode.estimatedCost = estimate
36     SORT joinNodes ascending by estimatedCost
37     REBUILD the join sequence left-deep, cheapest join first
38
39     RETURN tree // optimized, heuristically-ordered query tree
40
41
42 FUNCTION EstimateSize(node):
43     IF node is a leaf relation R: RETURN |R|
44     IF node is sigma_condition(child):
45         RETURN EstimateSize(child) * SelectivityOf(condition)
46     IF node is a join/product of (left, right):
47         RETURN EstimateSize(left) * EstimateSize(right)
48         * SelectivityOf(joinCondition)
```


Notes on the design

- The three rules are applied in this order deliberately: selections and projections must be pushed down first, because doing so shrinks every relation before the join-ordering step estimates their sizes - ordering joins on un-filtered relation sizes would give the wrong answer.
- `SelectivityOf(condition)` uses standard estimates (e.g., $1/|\text{distinct values}|$ for equality, a fixed fraction such as $1/3$ for range predicates when no histogram is available) so `EstimateSize` can be computed without actually running the query.
- Building the plan left-deep (rather than bushy) keeps the search space small and lets each join immediately reuse the previous join's pipelined output, which is why most heuristic optimizers default to it before a cost-based optimizer explores bushier plans.